



TECHNICAL OVERVIEW

LexoraAI

A retrieval-augmented generation system that answers questions about your documents — and verifies its own answers before showing them to you.

Every claim carries a citation back to a specific page. Claims that cannot be verified against a source are removed. When the documents do not contain the answer, the system says so instead of inventing one. This document explains what LexoraAI solves, why the usual approach falls short, and how each stage of the pipeline is built.

GENERATION

Groq

gpt-oss-20b + fallback chain

EMBEDDINGS

Jina AI

jina-embeddings-v4

RETRIEVAL

Hybrid

dense + BM25 → RRF → MMR

VERIFICATION

Per-claim

cite, score, abstain

Confident answers are not the same as correct ones

The organisations with the most documents are the ones least able to trust a language model with them, and for a specific reason.

A policy handbook, a set of annual reports, a stack of contracts — the value locked in them is not the text, it is the answers. Reading the text is a cost you pay to get an answer. Large language models look like the obvious remedy, and for a plausible-sounding paragraph they are. The trouble starts when the paragraph is wrong.

A model that has never seen your documents will still answer questions about them. It produces fluent, well-formatted, correctly-toned prose that is simply invented. In casual use that is annoying. In a compliance answer, a refund decision, or a clause interpretation, it is a liability — and because the wrong answer reads exactly like the right one, nothing in the output signals which you received.

Why retrieval alone does not fix it

Retrieval-augmented generation is the standard response: find relevant passages, put them in the prompt, instruct the model to use only those. It helps considerably. It does not close the gap, and the reasons are worth naming precisely, because each one is a design decision later in this document.

RECALL Retrieval quietly misses

If the passage holding the answer never reaches the prompt, no amount of prompt discipline recovers it. The model answers from its own weights instead — the exact failure the retrieval was supposed to prevent. Recall is the ceiling on everything downstream.

CHUNKING Boundaries destroy meaning

Slicing text every N tokens cuts sentences in half and mixes unrelated topics into a single vector. A chunk reading "it grew 14% year over year" is unfindable by "revenue growth" once the heading that gave it a subject is in a different chunk.

QUERIES The question is a bad query

Users write "and what about the second one?" A raw message carries unresolved pronouns, bundles two questions together, and is phrased as a question while the passage that answers it is phrased as a statement.

TRUST "Use only the context" is a request

It is an instruction, not a constraint. Models blend retrieved facts with remembered ones, cite passages that were never supplied, and swap a digit in a figure. Nothing in a standard RAG pipeline checks the output against its own sources.

THE PREMISE

An answer you cannot check is worth less than a citation you can. LexoraAI is built on the view that **the verification step is not an optional extra on top of RAG — it is the part that makes RAG usable**. Everything else in the system exists to give that step something good to work with.

Answer, cite, verify — or decline

Upload documents; ask questions in natural language; receive an answer where every claim points at a page, along with the evidence for trusting it.

The two outcomes that matter

Most RAG systems have one output shape: an answer. LexoraAI has two, and the second is the point.

```
// The system found it, and can prove where.
Q: What does resolution VX-2291 authorise?
A: Resolution VX-2291 authorises a share buyback of up to 15 million euros [S1].
  ✓ grounded · 90% confidence · hybrid retrieval · openai/gpt-oss-20b · 3.5s
  [S1] Vertex Labs Annual Review > Governance · p.3 · dense+sparse

// The system could not, and says so rather than guessing.
Q: Who is the CEO of Vertex Labs?
A: I could not verify an answer to this in your uploaded documents. The passages
  I retrieved touch on the topic but do not actually state the answer, so I
  would rather tell you that than guess.
  △ not found in sources · 15% confidence
```

The second response is not a failure mode being handled gracefully. It is a first-class result. A system that never abstains is not accurate — it is hallucinating and hiding it, and the evaluation harness measures abstention as its own metric for exactly that reason.

DESIGN NOTE — THE TRACE

Every stage is timed and reports what it decided. When an answer is wrong, the trace says whether retrieval missed the passage, the reranker buried it, or the model ignored it — three problems with three different fixes that are otherwise indistinguishable from the outside.

What a caller actually receives

Concretely, a query response reports not just the answer but how it was produced:

```
{
  "answer": "Q3 revenue was $48.2 million, growing 14.6% year over year [S1].",
  "confidence": 0.94,           // how well each claim is supported by its citation
  "grounded": true,           // every claim verified against a source
  "abstained": false,         // true when support was too weak to answer
  "degraded": false,          // true when a fallback path produced this
  "model": "openai/gpt-oss-20b",
  "plan": {
    "intent": "document_qa",
    "standalone_question": "...", // the history-resolved rewrite
    "search_queries": ["..."],    // decomposed sub-queries
    "hyde_used": true
  },
  "retrieval": { "mode": "hybrid", "dense_ok": true, "sparse_ok": true, "candidates": 24 },
  "grounding": { "support_ratio": 1.0, "unsupported_claims": 0, "invalid_citations": [] },
  "trace": { "total_ms": 3529, "stages": { "plan": 401, "retrieve": 1349, "rerank": 492,
                                           "generate": 613, "verify": 373 } },
  "sources": [
    { "section": "FY24 Report > Revenue", "page_number": 1, "cited": true,
      "relevance_score": 1.0, "dense_score": 0.78, "sparse_score": 1.0,
      "rerank_score": 1.0, "retrievers": ["dense", "sparse"] }
  ]
}
```

Two paths through one index

Ingestion writes to two indexes that answer different kinds of question. Answering reads from both, fuses them, then narrows.

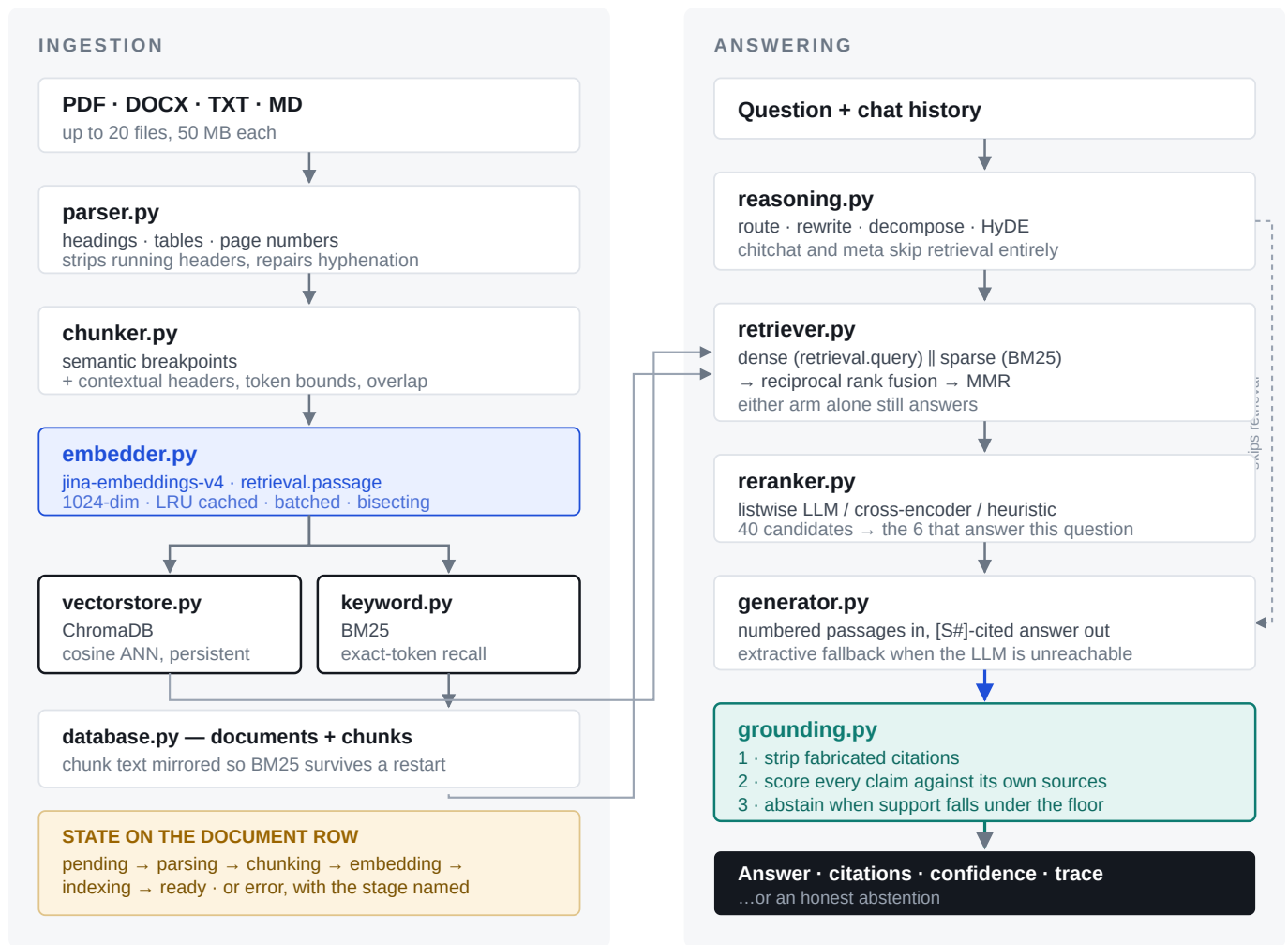


Figure 1 — System architecture. Ingestion (left) writes chunk vectors to ChromaDB and chunk text to SQL, from which BM25 is rebuilt on start-up. Answering (right) reads both indexes, fuses their rankings, narrows to a handful of passages, then checks the generated answer back against those same passages before returning it. Dashed lines are the paths that skip a stage.

Why the chunk text is stored twice

A vector store cannot serve lexical search — it holds vectors, and BM25 needs words. So every chunk's text is mirrored into a SQL `chunks` table. That single decision buys three things: BM25 rebuilds from durable storage after a restart, source snippets are returned without a second lookup, and the embedding model can be changed without re-uploading anything, because `scripts/reindex.py` re-embeds from the stored text.

Everything downstream inherits the chunks

Retrieval quality is capped by chunk quality, and chunk quality is capped by parse quality. Both stages are treated as first-class rather than as glue.

Parsing: keep the structure, drop the noise

A naive parser flattens a document into a wall of text, discarding the two signals retrieval cares about most: *where* a passage sits and *what section* it belongs to. LexoraAI's parser emits typed blocks instead — each tagged with a page number, a kind (`heading` / `body` / `table` / `list`) and its heading breadcrumb — while removing the artefacts that otherwise end up inside embeddings:

- **Repeated running headers and footers.** Detected by frequency across pages, so "Confidential — Internal Use Only" does not appear in forty separate chunks.
- **Line-break hyphenation.** `compre-\nhensive` is rejoined, so the token that matters is actually present for BM25.
- **Heading detection from fonts and styles.** PyMuPDF exposes size and weight; python-docx exposes style names. Both feed the same block model.

Chunking: semantic breakpoints and contextual headers

Slicing the token stream every *N* tokens is the single biggest source of bad retrieval. Two mechanisms replace it.

1 Semantic breakpoints

A boundary is placed where consecutive-sentence similarity drops below the 88th-percentile threshold — where the topic actually changes, rather than where the token counter ran out. Headings and tables are hard boundaries. Groups are then token-bounded: oversized ones split, undersized ones merged into a neighbour, and a sentence overlap stitched back on so no answer falls through a seam.

2 Contextual headers

Each chunk is *embedded* with its document title and heading breadcrumb prepended, while the *stored* text stays clean. A chunk reading "it grew 14% year over year" becomes findable by "revenue growth" because its embedded form carries `[Document: FY24 Report | Section: Revenue | Page: 12]` . The user never sees the header; the retriever always does.

FAILURE IS REPORTED PER STAGE

Ingestion runs in the background and writes its current stage onto the document row — `parsing → chunking → embedding → indexing → ready` . When something breaks, the API reports which stage broke rather than a generic error, and `GET /documents/{id}` can be polled for live progress.

Configuration that matters

SETTING	DEFAULT	EFFECT
CHUNK_STRATEGY	semantic	Semantic breakpoints, or <code>recursive</code> for purely structural grouping
CHUNK_SIZE_TOKENS	420	Upper bound per chunk; larger favours context, smaller favours precision
CHUNK_OVERLAP_TOKENS	64	Sentence overlap carried across a boundary, never across a section
SEMANTIC_BREAKPOINT_PERCENTILE	88	Higher splits less often, producing larger and more topical chunks
CONTEXTUAL_HEADERS	True	Prepend document and section to the embedded text only

Questions and passages are not the same kind of text

Embeddings are served by Jina AI's `jina-embeddings-v4`, chosen for two properties that a plain sentence encoder does not offer.

Groq — the generation provider — serves chat completions only; it has no embeddings API. That forces an explicit choice for the embedding half of the system, and the choice is `jina-embeddings-v4` called over HTTP, with a local ONNX model retained as an offline fallback.

Typed tasks: the asymmetry most pipelines lose

A question and the passage answering it are worded differently. Embedding them identically is a quiet, extremely common cause of lost recall. `jina-embeddings-v4` exposes a typed `task` parameter, and LexoraAI carries a *role* through every embedding call that maps directly onto it:

ROLE	JINA TASK	USED FOR
query	retrieval.query	The user's question, its decomposed sub-queries, and the HyDE probe
passage	retrieval.passage	Every indexed chunk, embedded with its contextual header
similarity	text-matching	Symmetric comparisons: sentence-to-sentence during semantic chunking, and claim-against-evidence during grounding

The role is part of the cache key, so the same sentence embedded as a query and as a passage cannot collide. The local `bge-small` fallback maps the same roles onto its instruction prefix; the ONNX last resort ignores them. Callers never branch on provider.

Matryoshka dimensions: a size dial, not a model swap

The model is Matryoshka-trained: its 2048-dimension output can be truncated to any shorter prefix and remains a valid embedding. `JINA_EMBED_DIMENSIONS=1024` therefore halves index size and cosine cost for roughly one percent of retrieval quality — no second model, no re-tuning, one environment variable.

Cached

A content-addressed LRU sits in front of the provider. Re-uploads, HyDE probes, MMR passes and grounding claims never pay for the same text twice.

Batched

Requests are grouped to `EMBEDDING_BATCH_SIZE`, with retries on 429 and 5xx using exponential backoff and jitter.

Bisecting

A failing batch is split in half recursively until the bad input is isolated, so one pathological chunk cannot sink an entire upload.

Changing the model safely

Vectors are only comparable to vectors from the same model. A silent model swap either errors on a dimension mismatch or, worse, returns nonsense at the same dimension. The vector store therefore writes an *embedding fingerprint* — provider, model and dimension — on first write and compares it on every later one, logging loudly on a mismatch. Recovery never requires a re-upload:

```
# re-embed every stored chunk with whatever EMBEDDING_PROVIDER now points at
PYTHONPATH=. python scripts/reindex.py --drop      # rebuild the index
PYTHONPATH=. python scripts/reindex.py --dry-run    # preview the work first
```

The question is not the search query

Retrieval quality is capped by whatever it is handed. One planning call reshapes the user's message into something worth searching with.

A raw user message carries pronouns that only resolve against the conversation, bundles two questions into one, and is phrased as a question while the passage that answers it is phrased as a statement. A single planning call to the LLM fixes all of that at once, returning a structured plan:

FIELD	WHAT IT DOES
intent	Routes the request. <code>chitchat</code> and <code>meta</code> ("what documents do I have?") skip retrieval entirely instead of hallucinating over noise
standalone_question	Resolves pronouns against chat history, so "and when does it expire?" becomes a self-contained query
search_queries	Decomposes into up to three keyword-shaped sub-queries, phrased the way a passage would be worded rather than as a question
hypothetical_answer	A HyDE probe — an invented answer embeds far closer to the real passage than the question does

Two safety nets keep the planner from becoming a single point of failure: obvious small talk is caught by a regex fast-path before any network call, and if the planning call fails, the pipeline falls back to a plan built from the raw question. Quality drops; answering continues.

A worked example

The hard case is the follow-up. The planner is given one explicit test — *does the message name its own subject?* If it does, it is a new question and is copied almost verbatim; if it is only a pronoun or a fragment, the missing subject is pulled from the history. That single rule prevents the classic failure where "and what about expenses?" becomes "What does STD-441 say about expenses?" and retrieves the wrong document entirely.

```
// history user: "What does policy STD-441 require?"
// message user: "and when does it expire?"           ← pronoun only

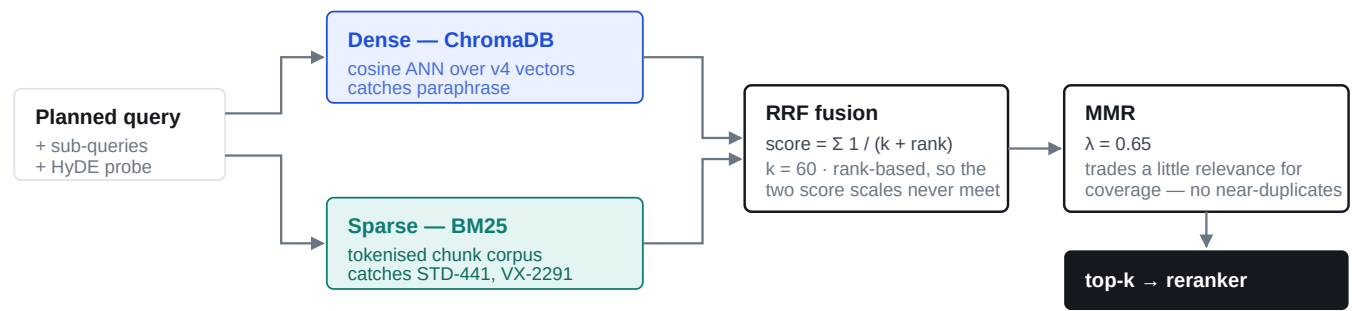
{
  "intent": "document_qa",
  "standalone_question": "When does policy STD-441 expire?",
  "search_queries": ["STD-441 expiry date", "policy STD-441 remains in force until"],
  "hypothetical_answer": "Policy STD-441 remains in force until 31 December 2026,
                          after which it must be reviewed and reissued.",
  "needs_retrieval": true
}
```

Note what the plan is optimised for. `search_queries` are phrased as a passage would word them, not as questions. `hypothetical_answer` is deliberately invented — it is never shown to anyone, it exists only to give the dense retriever a probe that sits in the same region of vector space as the real passage.

Two retrievers, fused by rank, then narrowed

Dense vectors are strong at paraphrase and weak at exact tokens. BM25 is the exact inverse. Running both is why hybrid retrieval beats either alone.

Ask for `STD-441` or `VX-2291` and embeddings will cheerfully return the semantically adjacent paragraph — it is *about* the same topic. BM25 returns the one that contains the string. Ask "how do I get my money back" and BM25 finds nothing, because the document says "refund". Neither retriever is redundant.



Each arm is independently fault-tolerant: if the vector store is down, BM25 still answers — and the reverse.

Figure 2 — Hybrid retrieval. Reciprocal Rank Fusion combines the two rankings by position rather than by score, so a cosine similarity and a BM25 score never have to be calibrated against one another. Passages found by *both* arms rise above passages found by only one.

Reranking: recall in, precision out

Retrieval optimises recall — pull 20 to 40 plausible chunks. Reranking optimises precision — decide which five or six actually answer *this* question, using a model that sees query and passage together rather than comparing two independent vectors. Fewer, better passages in the prompt is the cheapest hallucination reduction available. Three modes degrade into one another:

MODE	COST	NOTES
llm	~300 ms, no extra RAM	Listwise scoring by the Groq model. Default — free-tier instances have 512 MB
cross_encoder	~90 MB RAM, offline	Local ONNX cross-encoder; no network dependency at query time
heuristic	Free, no I/O	Lexical overlap, coverage and retrieval prior. Always available

Checking the output instead of trusting the prompt

Telling a model "only use the context" reduces hallucination. It does not eliminate it. This is the stage that distinguishes LexoraAI from a standard RAG pipeline.

- 1

Citation validity

Every `[S#]` marker in the answer is parsed. Any marker pointing at a passage number that was never in the prompt is a *fabricated citation* — the most damaging failure mode available, because it looks verified. Those are stripped before the answer is returned.
- 2

Per-claim support

The answer is split into claim sentences, and each is scored against *its own cited passages* — not against the pooled context, which would let one well-supported sentence launder an unsupported one. The signal is hybrid: embedding cosine catches paraphrase, while weighted lexical overlap leaning on numbers, dates and proper nouns catches the digit-swap error that embeddings miss entirely. "€15 million" and "€51 million" are neighbours in vector space.
- 3

Optional LLM adjudication

Claims landing in the ambiguous band between clearly supported and clearly unsupported can be sent for a second opinion from an NLI-style verifier call, enabled per request.
- 4

Abstention

If overall support falls under `ABSTAIN_THRESHOLD`, the answer is replaced with an honest statement that it could not be verified, rather than shipped in a confident tone. The confidence figure returned to the caller is this support score.

FAILS OPEN IN THE SAFE DIRECTION

If verification itself breaks — the embedding provider is unreachable, the verifier call times out — the answer is returned unmodified and flagged `verified: false`, rather than dropped. The caller is told the check did not run; they are never told an unchecked answer passed.

Grounding thresholds

SETTING	DEFAULT	EFFECT
ENABLE_GROUNDING_CHECK	True	Master switch for the whole verification stage
GROUNDING_MIN_SUPPORT	0.42	Per-claim threshold below which a claim counts as unsupported
ABSTAIN_THRESHOLD	0.25	Answer-level confidence floor; below it the system declines
STRIP_INVALID_CITATIONS	True	Remove <code>[S#]</code> markers that point at passages never supplied

Every dependency is assumed to fail

The system runs on a free-tier LLM API, a hosted embeddings API and a 512 MB instance. Degradation is designed rather than discovered.

FAILURE	WHAT HAPPENS
Model rate-limited (429)	Retries with exponential backoff, honouring Groq's own <code>retry-after</code> hint
Model down or decommissioned	Fails over to the next model in the configured chain
Repeated LLM failures	A circuit breaker opens; calls fail fast for a cooldown, then probe
LLM completely unreachable	Extractive fallback — the top passages are returned verbatim with their sources. Strictly worse than a written answer; still true, still cited, and better than a 502
Query planner fails	Falls back to the raw question. Quality drops, answering continues
Reranker fails	Degrades LLM → cross-encoder → heuristic lexical scorer
Jina rate-limits or 5xx	Retried with exponential backoff and jitter
Embedding provider missing	Degrades Jina → fastembed → Chroma's bundled MiniLM → OpenAI
One bad input in a batch	The batch is bisected so a single item cannot sink the rest
Vector store or BM25 down	The other retrieval arm still answers
Embedding model changed	Fingerprint mismatch is detected and logged loudly instead of silently poisoning retrieval; <code>scripts/reindex.py</code> re-embeds from stored chunk text

Observability

Three endpoints exist so that a misbehaving deployment can be diagnosed without attaching a debugger: `/health` is a fast liveness probe, `/health/deep` loads the embedding model and reports every component including circuit-breaker state, and `/stats` summarises the corpus and the active pipeline configuration. Every query response carries the per-stage trace shown in section 02.

What each choice buys

Selected for a system that must run credibly on a free tier while remaining honest about its own output.

LAYER	CHOICE	WHY
API	FastAPI + Uvicorn	Async, Pydantic validation, auto-generated OpenAPI docs at <code>/docs</code>
LLM	Groq <code>openai/gpt-oss-20b</code>	Very fast inference; <code>gpt-oss-120b</code> and <code>qwen3.6-27b</code> configured as fallbacks
Embeddings	Jina <code>jina-embeddings-v4</code>	Multilingual, long-context, typed query/passage tasks, Matryoshka dimensions. <code>bge-small</code> on ONNX is the offline fallback
Dense index	ChromaDB	Persistent cosine ANN with metadata filtering; no separate service to run
Sparse index	rank-bm25	Exact-token recall, rebuilt in memory from SQL on start-up
Reranking	Groq listwise / ONNX cross-encoder	Listwise costs no extra RAM; the cross-encoder runs fully offline
Database	SQLAlchemy — SQLite or PostgreSQL	Document metadata plus the durable BM25 corpus. SQLite locally, Postgres in Compose
Parsing	PyMuPDF, python-docx, tiktoken	Fonts and styles drive heading detection; tiktoken gives a universal token proxy
Frontend	Static HTML + optional Gradio	No build step, no bundler, no <code>node_modules</code>
Packaging	Docker, docker-compose, Render	Fallback embedding model baked into the image to avoid cold-start downloads
Testing	pytest — 142 tests	No API key and no network required; providers are stubbed at the boundary

Repository map

LexoraAI/

├─ app/

├─ main.py app, lifespan, errors

├─ config.py every knob, env-driven

├─ database.py documents + chunks

├─ rag/

├─ parser.py → structured blocks

├─ chunker.py → context-headed chunks

├─ embedder.py → vectors (Jina v4)

├─ vectorstore.py dense index (Chroma)

├─ keyword.py sparse index (BM25)

├─ retriever.py hybrid → RRF → MMR

├─ reranker.py precision pass

├─ reasoning.py plan · rewrite · HyDE

├─ generator.py cited answers

├─ grounding.py verification

├─ llm.py Groq client + breaker

├─ prompts.py all prompts

├─ pipeline.py orchestrator

├─ api/

├─ router.py mounts /api/v1

├─ upload.py ingestion endpoint

├─ query.py question answering endpoint

├─ documents.py list / get / delete

├─ system.py health, deep health, stats

├─ scripts/

├─ evaluate.py labelled evaluation harness

├─ reindex.py re-embed after a model change

├─ static/index.html web UI (no build step)

├─ frontend.py optional Gradio UI

├─ tests/ 142 tests

Endpoints and deployment

API surface

METHOD	ENDPOINT	WHAT IT DOES
GET	/api/v1/health	Fast liveness probe
GET	/api/v1/health/deep	Per-component status: circuit breaker, embedder, indexes
GET	/api/v1/stats	Corpus size and active pipeline configuration
POST	/api/v1/upload	Upload PDF / DOCX / TXT / MD — up to 20 at once, processed in the background
POST	/api/v1/query	Ask a question, optionally scoped to specific documents and carrying chat history
GET	/api/v1/documents	List documents
GET	/api/v1/documents/{id}	One document, including live ingestion <code>stage</code>
DELETE	/api/v1/documents/{id}	Remove it from the database, disk, vector index and BM25 corpus

Running it

LOCAL

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

cp .env.example .env
# GROQ_API_KEY — console.groq.com/keys
# JINA_API_KEY — jina.ai/embeddings

PYTHONPATH=. uvicorn app.main:app \
  --reload --port 8000
```

Web UI at `:8000`, Swagger at `/docs`. Nothing to download — embeddings are served over HTTP.

DOCKER

```
cp .env.example .env
docker-compose up --build -d
docker-compose logs -f api
```

PostgreSQL instead of SQLite; the fallback embedding model is baked into the image so a Jina outage costs no cold-start download. `render.yaml` deploys the same image to Render with a mounted disk.

TESTS & EVALUATION

```
PYTHONPATH=. pytest tests/ -q
PYTHONPATH=. python scripts/evaluate.py
PYTHONPATH=. python scripts/evaluate.py --compare
```

Measured, not asserted

A RAG system reports a single accuracy number only when it is hiding something. These are the figures that separate a working pipeline from a confident one.

Measured results

`scripts/evaluate.py` runs a labelled question set — 15 questions, 12 answerable and 3 deliberately unanswerable — and reports the metrics that actually matter for RAG rather than a single accuracy number.

METRIC	RESULT		WHAT IT MEANS
Recall@6	100%		The passage containing the answer was retrieved every time
MRR	1.00		It was ranked first every time
Answer accuracy	100%		The expected fact appeared in the answer
Groundedness	100%		Every answer passed citation verification
Correct abstention	100%		All three unanswerable questions were declined, not hallucinated
Latency p50	~4.0 s		End to end: planning, retrieval, reranking, generation, verification

READ THIS HONESTLY

The built-in corpus is small by design, so `precision@k` is capped by construction and these figures demonstrate that the pipeline works end to end — not that it will score 100% on your documents. Point `--dataset` at your own labelled set for a figure that means something about your corpus. `--compare` reruns the same set against hybrid, dense-only and sparse-only retrieval, so the value of the fusion is measured rather than asserted.

Cost and rate-limit note

On Groq's free tier the pipeline makes up to four LLM calls per question — plan, rerank, answer, optional verify — which can brush the 8,000 tokens-per-minute limit. Retry and fallback handle this transparently; setting `ENABLE_QUERY_PLANNING=False` and `RERANKER_MODE=heuristic` reduces it to a single call for anyone who would rather avoid the backoff. Embeddings are billed separately by Jina and are cached aggressively, so a re-upload of unchanged text costs nothing.

The through-line

SOLVES

Getting trustworthy answers out of a document corpus without reading it, and without having to independently verify every answer the system produces.

HOW

Structure-preserving parsing and semantic chunking, hybrid dense + sparse retrieval fused by rank, a precision reranking pass, then per-claim verification of the generated answer against the passages it cites.

WHY

Because a confident wrong answer is worse than no answer, and because nothing in a standard RAG pipeline distinguishes the two. The verification stage is the product; the rest of the pipeline exists to give it good material.

COST OF IT

Roughly four LLM calls and one embedding call per question, and an explicit refusal to answer when support is weak. Both are deliberate trades against the alternative.

Each design decision in this document traces back to one of the four failure modes named in section 01. Chunking with contextual headers answers "boundaries destroy meaning". Hybrid retrieval and HyDE answer "retrieval quietly misses". Query planning answers "the question is a bad query". Grounding answers "'use only the context' is a request, not a constraint". The typed embedding roles introduced in section 05 sharpen the first two at once, by ensuring a question and the passage that answers it are never embedded as though they were the same kind of text.

WHERE TO LOOK NEXT

`app/rag/pipeline.py` is the orchestrator and the only module the API layer talks to — it is the shortest path to understanding the whole system. `app/rag/grounding.py` is where the distinguishing behaviour lives. `app/config.py` lists every knob, all of them environment-driven.